

```
1 #include <assert.h>
2 #include <stdbool.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include "group.h"
7 #include "maps.h"
8 #include "linked_list.h"
9
10 int *zero_array(int n) {
11     int *res = malloc(n * sizeof(int));
12     for (int i = 0; i < n; i++) {
13         res[i] = 0;
14     }
15     return res;
16 }
17
18 void group_name(group_t *g, char *s) {
19     g->name = s;
20 }
21
22 void add_sum_to_decomp(int x, int y, decomp_t* d) {
23     int* x_decomp = d->basis_decomps[x];
24     int* y_decomp = d->basis_decomps[y];
25     int z = (x+y) % d->n;
26     int* z_decomp = d->basis_decomps[z];
27     for (int i = 0; i < d->n; i++) {
28         if (x_decomp[i] == -1 || y_decomp[i] == -1) {
29             z_decomp[i] = -1;
30         } else {
31             z_decomp[i] = (x_decomp[i] + y_decomp[i]) % d->n;
32         }
33     }
34 }
35
36 group_t *group_empty(int n) {
37     group_t *new_empty_group = malloc(sizeof(group_t));
38     new_empty_group->n = n;
39     new_empty_group->table = malloc(n * sizeof(int *));
40     for (int i = 0; i < n; i++) {
41         new_empty_group->table[i] = zero_array(n);
42     }
43     return new_empty_group;
44 }
45
46 void group_free(group_t *g) {
47     assert(g != NULL && "[ ] Pointeur nul.\n");
48     int n = g->n;
49     for (int i = 0; i < n; i++) {
50         free(g->table[i]);
51     }
52     free(g->table);
53     free(g);
54 }
55
56
57 gdata_t* create_empty_gdata() {
58     gdata_t* gdata = malloc(sizeof(gdata_t));
59     return gdata;
60 }
```

```
61
62 // WE GIVE THE POINTERS WE DON'T COPY.
63 gdata_t* parse_entries(list_t* gen, gentabl_t* rel) {
64     gdata_t* gdata = create_empty_gdata();
65     gdata->gen = gen;
66     gdata->rel = rel;
67     return gdata;
68 }
69
70 void free_gdata(gdata_t* gdata) {
71     assert(gdata != NULL);
72     free_list(gdata->gen);
73     free_gentabl(gdata->rel);
74     free(gdata);
75 }
76
77 list_t* get_generator(gdata_t* gdata) {
78     assert(gdata != NULL);
79     return gdata->gen;
80 }
81
82 gentabl_t* get_rel(gdata_t* gdata) {
83     assert(gdata != NULL);
84     return gdata->rel;
85 }
86
87 void group_print(group_t *g) {
88     assert(g != NULL && "[_] Pointeur nul.\n");
89     int n = g->n;
90     if (g->name == NULL || g->name[0] == '\0') {
91         printf("Groupe d'ordre: %d\n", n);
92     } else {
93         printf("Groupe %s d'ordre %d\n", g->name, n);
94     }
95     for (int i = 0; i < n; i++) {
96         for (int j = 0; j < n; j++) {
97             printf("%d ", g->table[i][j]);
98         }
99         printf("\n");
100     }
101 }
102
103 group_t *from_matrix(int **m) {
104     assert(m != NULL && "[_] Pointeur nul.\n");
105     int n = sizeof(*m[0]) / sizeof(int);
106     group_t *res = group_empty(n);
107     for (int i = 0; i < n; i++) {
108         for (int j = 0; j < n; j++) {
109             res->table[i][j] = m[i][j];
110         }
111     }
112     return res;
113 }
114
115 group_t *copy_group(group_t *g) {
116     assert(g != NULL && "[_] Pointeur nul.\n");
117     int n = g->n;
118     group_t *copy = group_empty(n);
119     for (int i = 0; i < n; i++) {
120         for (int j = 0; j < n; j++) {
```

```
121     copy->table[i][j] = g->table[i][j];
122 }
123 }
124 return copy;
125 }
126
127 int calculate(group_t* g, int x, int y) {
128     if (x == -1) {
129         return y;
130     }
131     return g->table[x][y];
132 }
133
134 bool is_neutral(group_t* g, int x) {
135     assert(g != NULL);
136     assert(x >= 0 && x < g->n);
137     int res;
138     if (x < g->n - 1) {
139         res = calculate(g, x, x+1);
140         return (res == x+1);
141     } else {
142         res = calculate(g, x, 0);
143         return (res == 0);
144     }
145 }
146
147 int get_neutral(group_t* g) {
148     assert(g != NULL);
149     for (int i = 0; i < g->n; i++) {
150         if (is_neutral(g, i)) {
151             return i;
152         }
153     }
154     printf("Error\n.");
155     return -1;
156 }
157
158 int calculate_order(group_t* g, int x) {
159     assert(g != NULL);
160     assert(x >= 0 && x < g->n);
161     if (is_neutral(g, x)) {
162         return 0;
163     }
164     int curr = x;
165     int order = 1;
166     while (!is_neutral(g, curr)) {
167         curr = calculate(g, curr, x);
168         order++;
169     }
170
171     return order;
172 }
173
174 int* get_orders_list(group_t* g) {
175     assert(g != NULL);
176     int* orders = malloc(g->n*sizeof(int));
177     for (int x = 0; x < g->n; x++) {
178         orders[x] = calculate_order(g, x);
179     }
180     return orders;
```

```
181 }
182
183 list_t* get_cyclic_subgroup_element(group_t* g, int x, int e) {
184     assert(g != NULL);
185     assert(0 <= x < g->n);
186     assert(is_neutral(g, e));
187     int curr = x;
188     list_t* cyclic = create_empty_list();
189     push(cyclic, e);
190     while (!is_neutral(g, curr)) {
191         push(cyclic, curr);
192         curr = calculate(g, curr, x);
193     }
194
195     return cyclic;
196 }
197
198 int calc_generated_subgroup(group_t* g, int new_gen, list_t* generators, gentabl_t*
decomp_tbl) {
199     assert(decomp_tbl->size == g->n);
200     assert(generators != NULL && g != NULL);
201
202     if (new_gen >= g->n) {
203         return -1;
204     }
205
206     push(generators, new_gen);
207
208     list_t* single = create_single(new_gen);
209     copy_at_index(decomp_tbl, new_gen, single);
210     free_list(single);
211
212     // Variable pour le candidat d'un autre générateur
213     int next_gen = new_gen+1;
214
215     int to_add;
216     list_t* to_add_decomp;
217     for (int elt = 0; elt < g->n; elt++) {
218         if (!is_empty_index(decomp_tbl, elt)) {
219             to_add = calculate(g, elt, new_gen);
220             if (is_empty_index(decomp_tbl, to_add)) {
221                 // Calculer la décomposition de to_add.
222                 to_add_decomp = get_copy_index_val(decomp_tbl, elt);
223                 append(to_add_decomp, new_gen);
224                 copy_at_index(decomp_tbl, to_add, to_add_decomp);
225
226                 free_list(to_add_decomp);
227             }
228             if (next_gen < g->n && !is_empty_index(decomp_tbl, next_gen)) {
229                 next_gen++;
230             }
231         }
232     }
233
234     return next_gen;
235 }
236
237 gdata_t* calc_gen_and_rel(group_t* g) {
238     list_t* generators = create_empty_list();
239     gentabl_t* relations = create_empty_gentabl(g->n);
```

```
240
241 int curr_gen = 0;
242 while (curr_gen < g->n) {
243     curr_gen = calc_generated_subgroup(g, curr_gen, generators, relations);
244 }
245
246 gdata_t* result = parse_entries(generators, relations);
247 return result;
248 }
249
250 void print_array(int* arr, int n) {
251     for (int i = 0; i < n; i++) {
252         printf("%d ", arr[i]);
253     }
254     printf("\n");
255 }
256
257 bool are_isomorphic(group_t* g1, group_t* g2) {
258     assert(g1 != NULL && g2 != NULL);
259     assert(g1->n == g2->n);
260
261     // Précalculs sur les données de g1.
262     gdata_t* g1_gdata = calc_gen_and_rel(g1);
263     gentabl_t* rel = g1_gdata->rel;
264     list_t* gen = g1_gdata->gen;
265
266     int n = g1->n;
267     int m = gen->length;
268
269     int count_perm = 0;
270     int* perm;
271     // Nombres de permutations.
272     llong num_perm = factorial(m);
273
274     // Table des coefficients binomiaux: binom[k][p] = k parmi p
275     int** binom = get_binomial(n);
276
277     int count = 0;
278     int curr_subset_num = 0;
279     int num_subsets = binom[m][n];
280     int* induced_map;
281     bool found = false;
282
283     while (curr_subset_num < num_subsets && !found) {
284         int* chosen_images = get_array_from_subset_binom(curr_subset_num, m, n, binom);
285         print_array(chosen_images, m);
286         int* curr_permutated_images;
287         int curr_perm_num = 0;
288         while (curr_perm_num < num_perm && !found) {
289             perm = get_kth_permutation_rec(m, curr_perm_num);
290             curr_permutated_images = get_from_permutation(perm, chosen_images, m);
291             induced_map = get_full_map(g1_gdata, g2, curr_permutated_images, m);
292             if (is_isomorphism(g1, g1_gdata, g2, induced_map, n)) {
293                 found = true;
294                 printf("[+] Isomorphisme trouvé.\n");
295                 print_map(induced_map, n);
296
297                 free(induced_map);
298                 free(perm);
299                 free(curr_permutated_images);
```

```
300         break;
301     }
302
303     free(induced_map);
304     free(perm);
305     free(curr_permutated_images);
306     curr_perm_num++;
307     count++;
308 }
309
310     free(chosen_images);
311     curr_subset_num++;
312 }
313
314     free_gdata(g1_gdata);
315     free_matrix(binom, n);
316
317     return found;
318
319 }
320
321 bool are_isomorphic_with_order(group_t* g1, group_t* g2) {
322     assert(g1 != NULL && g2 != NULL);
323     assert(g1->n == g2->n);
324
325     gdata_t* g1_gdata = calc_gen_and_rel(g1);
326     gentabl_t* rel = g1_gdata->rel;
327     int* orders_g1 = get_orders_list(g1);
328     int* orders_g2 = get_orders_list(g2);
329
330     list_t* gen = g1_gdata->gen;
331
332     int n = g1->n;
333     int m = gen->length;
334
335     int count_perm = 0;
336     int* perm;
337     long num_perm = factorial(m);
338
339     int** binom = get_binomial(n);
340
341     int count = 0;
342
343     int curr_subset_num = 0;
344     int num_subsets = binom[m][n];
345     int* induced_map;
346     bool found = false;
347
348     while (curr_subset_num < num_subsets && !found) {
349         int* chosen_images = get_array_from_subset_binom(curr_subset_num, m, n, binom);
350         int* curr_permutated_images;
351         int curr_perm_num = 0;
352         while (curr_perm_num < num_perm && !found) {
353             perm = get_kth_permutation_rec(m, curr_perm_num);
354             curr_permutated_images = get_from_permutation(perm, chosen_images, m);
355             induced_map = get_full_map(g1_gdata, g2, curr_permutated_images, m);
356             if (is_isomorphism_with_order(g1, g1_gdata, g2, orders_g1, orders_g2,
357 induced_map, n)) {
357                 found = true;
358             }
359         }
360     }
```

```

359         free(induced_map);
360         free(perm);
361         free(curr_permutated_images);
362         break;
363     }
364
365     free(induced_map);
366     free(perm);
367     free(curr_permutated_images);
368     curr_perm_num++;
369     count++;
370 }
371
372     free(chosen_images);
373     curr_subset_num++;
374 }
375
376 free(orders_g1);
377 free(orders_g2);
378 free_gdata(g1_gdata);
379 free_matrix(binom, n);
380
381 return found;
382 }
383
384 bool are_isomorphic_with_order_custom_gen(group_t* g1, group_t* g2, int* generators,
int num_gen) {
385     assert(g1 != NULL && g2 != NULL);
386     assert(g1->n == g2->n);
387     assert(generators != NULL && num_gen > 0);
388
389     list_t* gen = create_empty_list();
390     gentabl_t* rel = create_empty_gentabl(g1->n);
391
392     for (int i = 0; i < num_gen; i++) {
393         assert(generators[i] >= 0 && generators[i] < g1->n);
394         calc_generated_subgroup(g1, generators[i], gen, rel);
395     }
396
397     gdata_t* g1_gdata = create_empty_gdata();
398     g1_gdata->gen = gen;
399     g1_gdata->rel = rel;
400
401     int* orders_g1 = get_orders_list(g1);
402     int* orders_g2 = get_orders_list(g2);
403
404     int n = g1->n;
405     int m = gen->length;
406
407     int* perm;
408     llong num_perm = factorial(m);
409
410     int** binom = get_binomial(n);
411
412     int count = 0;
413     int curr_subset_num = 0;
414     int num_subsets = binom[m][n];
415     int* induced_map;
416     bool found = false;
417

```

```

418 while (curr_subset_num < num_subsets && !found) {
419     int* chosen_images = get_array_from_subset_binom(curr_subset_num, m, n, binom);
420     int* curr_permutated_images;
421     int curr_perm_num = 0;
422
423     while (curr_perm_num < num_perm && !found) {
424         perm = get_kth_permutation_rec(m, curr_perm_num);
425         curr_permutated_images = get_from_permutation(perm, chosen_images, m);
426         induced_map = get_full_map(g1_gdata, g2, curr_permutated_images, m);
427
428         if (is_isomorphism_with_order(g1, g1_gdata, g2, orders_g1, orders_g2,
induced_map, n)) {
429             found = true;
430             printf("[+] Isomorphisme trouvé.\n");
431             print_map(induced_map, n);
432
433             free(induced_map);
434             free(perm);
435             free(curr_permutated_images);
436             break;
437         }
438
439         free(induced_map);
440         free(perm);
441         free(curr_permutated_images);
442         curr_perm_num++;
443         count++;
444     }
445
446     free(chosen_images);
447     curr_subset_num++;
448 }
449
450 free(orders_g1);
451 free(orders_g2);
452 free_gdata(g1_gdata);
453 free_matrix(binom, n);
454
455 return found;
456 }
457
458 bool are_isomorphic_random(group_t* g1, group_t* g2) {
459     assert(g1 != NULL && g2 != NULL);
460     assert(g1->n == g2->n);
461
462     gdata_t* g1_gdata = calc_gen_and_rel(g1);
463     gentabl_t* rel = g1_gdata->rel;
464     int* orders_g1 = get_orders_list(g1);
465     int* orders_g2 = get_orders_list(g2);
466
467     list_t* gen = g1_gdata->gen;
468
469     int n = g1->n;
470     int m = gen->length;
471
472     int count_perm = 0;
473     int* perm;
474     llong num_perm = factorial(m);
475
476     int** binom = get_binomial(n);

```



```
477
478 int count = 0;
479
480 int curr_subset_num = 0;
481 int num_subsets = binom[m][n];
482 int* induced_map;
483 bool found = false;
484
485 while (curr_subset_num < num_subsets && !found) {
486     int* chosen_images = get_array_from_subset_binom(curr_subset_num, m, n, binom);
487     int* curr_permutated_images;
488     int curr_perm_num = 0;
489     while (curr_perm_num < num_perm && !found) {
490         perm = get_kth_permutation_rec(m, curr_perm_num);
491         curr_permutated_images = get_from_permutation(perm, chosen_images, m);
492         induced_map = get_full_map(g1_gdata, g2, curr_permutated_images, m);
493         if (is_isomorphism_with_order(g1, g1_gdata, g2, orders_g1, orders_g2,
induced_map, n)) {
494             found = true;
495             printf("[+] Isomorphisme trouvé.\n");
496             print_map(induced_map, n);
497
498             free(induced_map);
499             free(perm);
500             free(curr_permutated_images);
501             break;
502         }
503
504         free(induced_map);
505         free(perm);
506         free(curr_permutated_images);
507         curr_perm_num++;
508         count++;
509     }
510
511     free(chosen_images);
512     curr_subset_num++;
513 }
514
515 free(orders_g1);
516 free(orders_g2);
517 free_gdata(g1_gdata);
518 free_matrix(binom, n);
519
520 return found;
521 }
522
523 bool are_isomorphic_probabilistic(group_t* g1, group_t* g2, int max_attempts) {
524     assert(g1 != NULL && g2 != NULL);
525     assert(g1->n == g2->n);
526
527     srand(time(NULL));
528
529     gdata_t* g1_gdata = calc_gen_and_rel(g1);
530     gentabl_t* rel = g1_gdata->rel;
531     list_t* gen = g1_gdata->gen;
532
533     int n = g1->n;
534     int m = gen->length;
535 }
```

```
536 int* orders_g1 = get_orders_list(g1);
537 int* orders_g2 = get_orders_list(g2);
538
539 int* gen_orders = malloc(m * sizeof(int));
540 int gen_idx = 0;
541 for (node_t* c = gen->head; c != NULL; c = c->next) {
542     gen_orders[gen_idx++] = orders_g1[c->val];
543 }
544
545 bool found = false;
546 int* induced_map = NULL;
547
548 for (int attempt = 0; attempt < max_attempts && !found; attempt++) {
549     int* chosen_images = get_random_subset(m, n);
550
551     bool order_compatible = true;
552     for (int i = 0; i < m; i++) {
553         if (gen_orders[i] != orders_g2[chosen_images[i]]) {
554             order_compatible = false;
555             break;
556         }
557     }
558
559     if (!order_compatible) {
560         free(chosen_images);
561         continue;
562     }
563
564     int* perm = get_random_permutation(m);
565     int* permuted_images = get_from_permutation(perm, chosen_images, m);
566
567     induced_map = get_full_map(g1_gdata, g2, permuted_images, m);
568
569     if (is_isomorphism_with_order(g1, g1_gdata, g2, orders_g1, orders_g2,
induced_map, n)) {
570         found = true;
571         printf("[+] Isomorphisme trouvé après %d itérations!\n", attempt + 1);
572         print_map(induced_map, n);
573     }
574
575     free(chosen_images);
576     free(perm);
577     free(permuted_images);
578     if (!found) {
579         free(induced_map);
580     }
581
582     if ((attempt + 1) % 10000 == 0) {
583         printf("Itération %d...\n", attempt + 1);
584     }
585 }
586
587 free(gen_orders);
588 free(orders_g1);
589 free(orders_g2);
590 free_gdata(g1_gdata);
591
592 if (found) {
593     free(induced_map);
594 }
```

```
595
596     return found;
597 }
598
599 bool are_isomorphic_hybrid(group_t* g1, group_t* g2, long long
deterministic_threshold, int probabilistic_attempts) {
600     assert(g1 != NULL && g2 != NULL);
601     assert(g1->n == g2->n);
602
603     gdata_t* g1_gdata = calc_gen_and_rel(g1);
604     int n = g1->n;
605     int m = g1_gdata->gen->length;
606     free_gdata(g1_gdata);
607
608     int** binom = get_binomial(n);
609     long long num_subsets = binom[m][n];
610     free_matrix(binom, n);
611
612     llong num_perm = factorial(m);
613     long long total_candidates = num_subsets * num_perm;
614
615     if (total_candidates <= deterministic_threshold) {
616
617         return are_isomorphic_with_order(g1, g2);
618     } else {
619         return are_isomorphic_probabilistic(g1, g2, probabilistic_attempts);
620     }
621 }
622
623 bool are_isomorphic_adaptive(group_t* g1, group_t* g2) {
624     assert(g1 != NULL && g2 != NULL);
625     assert(g1->n == g2->n);
626
627     int n = g1->n;
628     gdata_t* g1_gdata = calc_gen_and_rel(g1);
629     int m = g1_gdata->gen->length;
630     free_gdata(g1_gdata);
631
632     int** binom = get_binomial(n);
633     long long total_space = binom[m][n] * factorial(m);
634     free_matrix(binom, n);
635
636     int attempts = (int)sqrt((double)total_space);
637     if (attempts < 1000) attempts = 1000;
638     if (attempts > 1000000) attempts = 1000000;
639
640     return are_isomorphic_probabilistic(g1, g2, attempts);
641 }
```